

音ゲー好き プログラマー

名前

ねぎし あゆむ
根岸 歩夢

学校

大分情報ITクリエイター専門学校

趣味

音ゲー、ボカロを聴く、アニメ鑑賞

GitHub URL

<https://github.com/Ayu20060301>

スキル

Visual Studio



C/C++ 2年
C# 1年半

Unity



1年半

フォトショップ



2年

Blender



1年

Git



1年



目次

p3

p4～p21

p22～p27

p28

自己PR

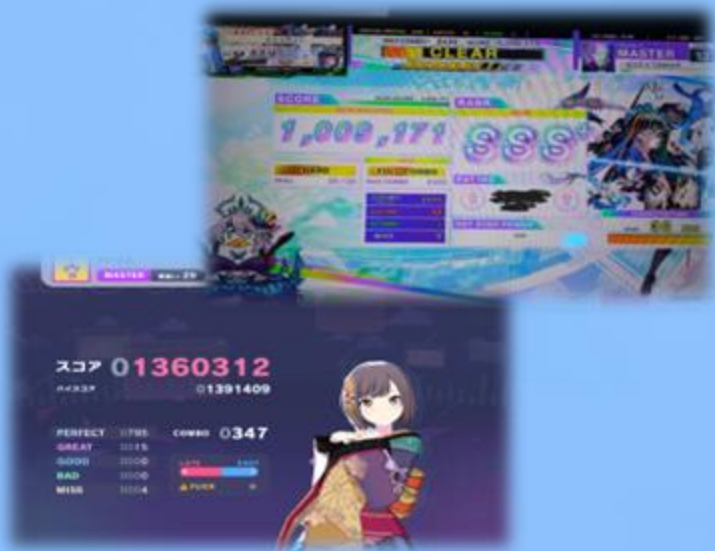
就活作品

その他制作物

今後の目標



自己PR



音ゲーが大好きです!

自作で音ゲーも作ってます!



就活作品



就活作品

作品名 : Rhythm Strike!!

ジャンル : 3Dリズムアクション

開発環境 : C#,Unity

制作期間 : 2025年9月～2026年2月

制作人数 : 1人

GitURL : <https://github.com/Ayu20060301/Rhythm-Strike>

動画URL : <https://youtu.be/3ogXOeafxxk>



作品概要



ジャンル

リズム



アクション



リズムに乗って
敵や障害物を越えよう!

このゲームの面白さ①

攻撃

Rhythm
Stroke!!

- ・ リズムに合わせた攻撃アクション
- ・ ノーツをタイミングよく押して敵を攻撃
- ・ コンボをつなぐ爽快感



このゲームの面白さ②

- ・ 障害物を避けるアクション

- ・ ジャンプで障害物を回避



← ジャンプ

避ける→



技術紹介

```
... in Input.Json\n\nnoteData = DataBaseManager\nnoteData == null)\n\nDebug.LogWarning($"NoteData\ncontinue;\n\n(番号 / LPB) * 小節の長\nme =\nm * (60.0f / inputJson.BPM\nson.offset * 0.001f;\n\n追加\nm.Add(note.block);\nType.Add(note.type);\nTime.Add(time);\n\nD座標(時間 ×\ntime * _notesSp\n\nInstantiate(\nData.prefab,\nVector3(note.block -\nData.pr... transform.rot
```

```
データベースを一括管理するクラス\n\nManager : SingletonMonoBehaviour<\n\nデータベースを入れる")]\nsDataBase noteDataBase; //ノーツ関連のデータ\nDataBase soundDataBase; //サウンド関連のデ\nDataBase uiDataBase; //UI関連のデータ\nDataBase songDataBase; //曲関連のデータ\nDataBase particleDataBase; //パーティ\nDataBase movieDataBase; //Movie関連のデ\n\nグローバル読み込み可能\n\nDataBase noteDB => noteDataBase;\n\nDataBase soundDB => soundDataBase;\n\nDataBase uiDB => uiDataBase;\n\nDataBase songDB => songDataBase;\n\nParticleDataBase particleDB => particleDataBase;\n\nDataBase movieDB => mov\n\n()
```

技術紹介① : 譜面作成 ※使用したNoteEditor



➡ <https://github.com/setchi/NoteEditor>

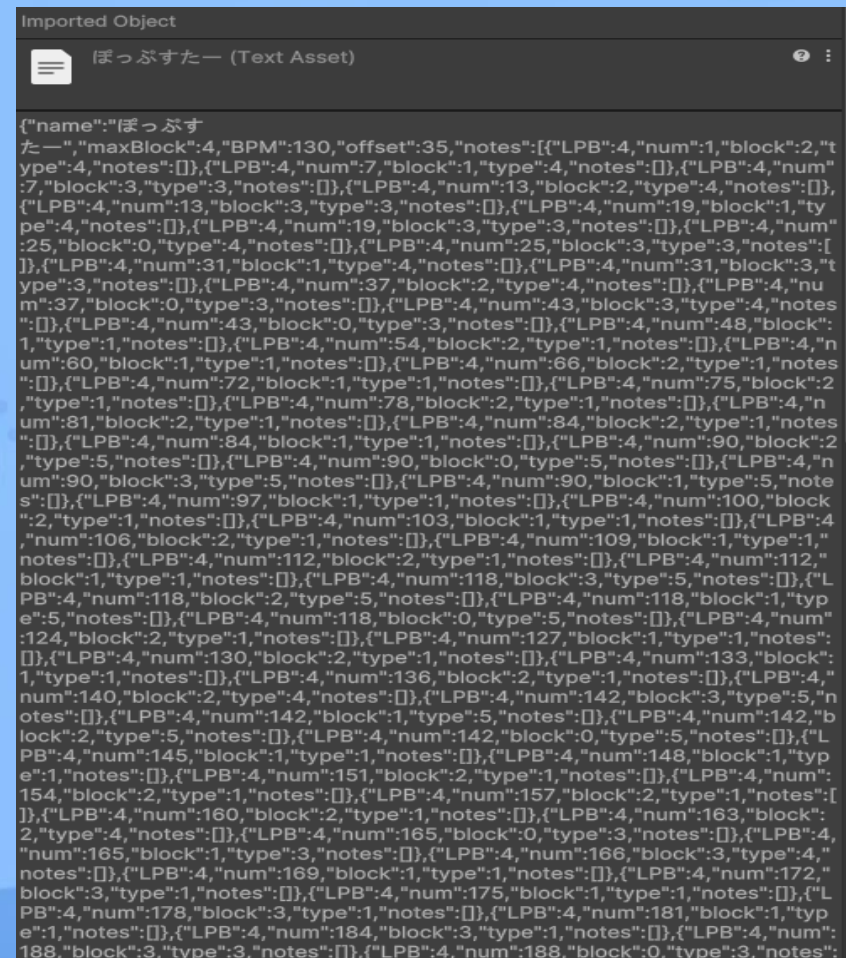
NoteEditorという外部Editorを使用して
譜面を作成しました。

下の画像のEditorで譜面の情報を入れ
Jsonファイルで保存されます。

▼実際に使用しているEditor



▼読み込んだJsonファイルの中身



技術紹介① : NoteEditorの拡張

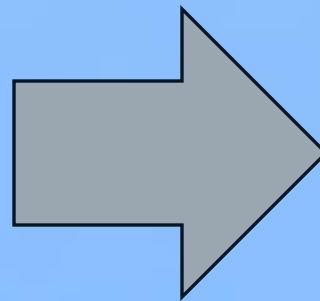


既存のNoteEditorでは配置できるノーツの種類が限定されていたため、**Editorの改良**を行い、新たなノーツタイプを追加できる仕組みを実装しました。

これにより**譜面制作の自由度**を向上させました。

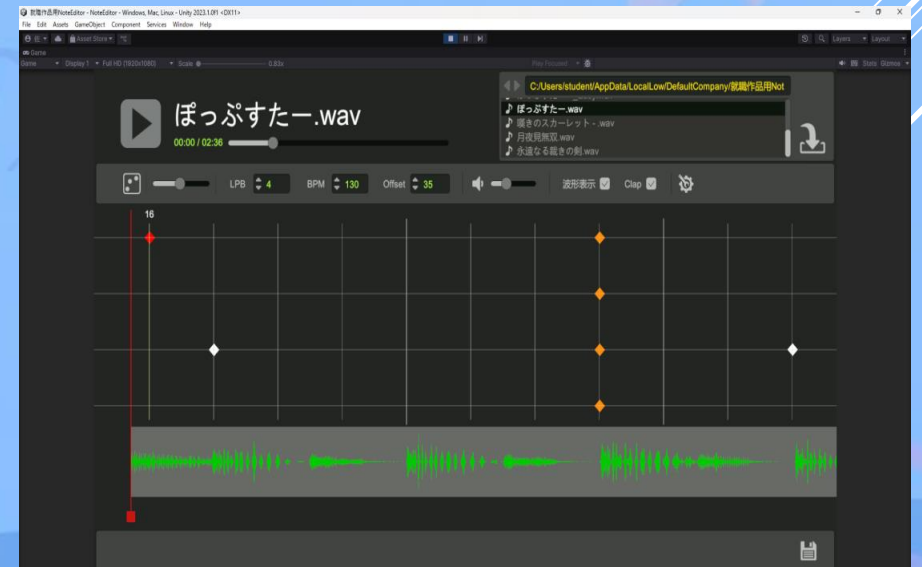
▼元のNoteEditor

☆ノーツの種類が2種類しかなかった



▼改良後のNoteEditor

★ノーツの種類が5種類に増えた



技術紹介② : ノーツの生成 (1 / 2)



Jsonファイルから曲ごとのノーツ情報を読み込み、判定線に到達する時間を計算してPrefabを生成するクラスを作成しました。

ノーツの種類・レーン・判定時間を管理することで、スコアやコンボ計算に必要な情報を提供しています。

▼ NotesManager.cs 参照

```
float secPerBeat = 60.0f / inputJson.BPM; //1拍
float secPerBar = secPerBeat * 4.0f; //1小節(4拍)

foreach(var note in inputJson.notes)
{
    var noteData = DataBaseManager.instance.noteDataBase.GetNoteByData(note.type);
    if(noteData == null)
    {
        Debug.LogWarning($"NoteDataが見つかりません type={note.type}");
        continue;
    }

    //time = (番号 / LPB) * 小節の長さ + offset
    float time =
        note.num * (60.0f / inputJson.BPM / note.LPB)
        + inputJson.offset * 0.001f;

    //リストに追加
    laneNum.Add(note.block);
    noteType.Add(note.type);
    notesTime.Add(time);

    //ノーツのZ座標(時間 × 速度)
    float z = time * _notesSpeed;

    var obj = Instantiate(
        noteData.prefab,
        new Vector3(note.block - 1.5f, noteData.y, z),
        noteData.prefab.transform.rotation
    );

    notesObj.Add(obj);
}
```


技術紹介② : ノーツの生成 (2 / 2)



Jsonデータに基づき、各ノーツが正しいレーン・タイミングで生成されます。

ノーツの種類ごとにPrefabを使い分け、判定線に到達するタイミングまで正確に落下します。

▼生成されたノーツ



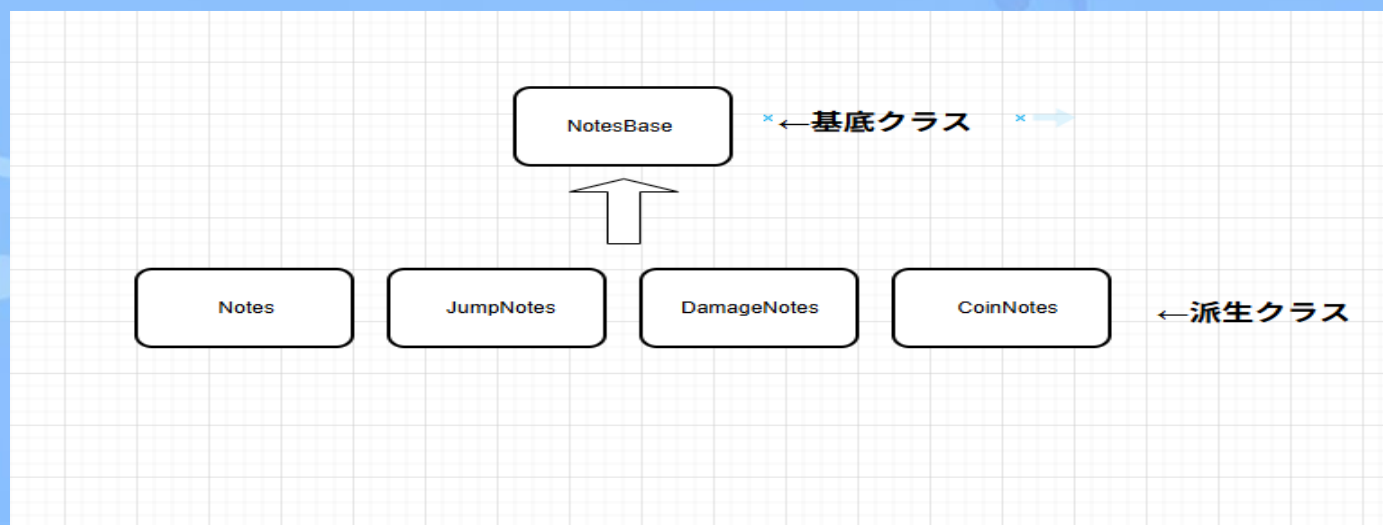
技術紹介③ : ノーツの基底クラス(NotesBase)

ノーツ共通の挙動をまとめた基底クラスを実装しました。

ノーツごとのクラス分けを行ったことにより、可読性と
管理のしやすさが向上しています。

また、今後のノーツ追加や変更にも柔軟に対応できる拡張性の
高い設計になっています。

▼簡易なクラス図



技術紹介④ : ノーツ判定システム



プレイヤー入力に応じたノーツ判定を管理するクラスを実装しました。

Critical、Hit、Attackの判定をms単位で制御するようにしています。

▼判定の許容時間

```
/// <summary>
/// ノーツ判定を管理するクラス
/// </summary>
// Unity スクリプト (1 件のアセット参照) 10 個の参照
public class Judge : MonoBehaviour
{
    //判定の許容時間
    private const float _kCriticalTime = 0.025f; //25ms
    private const float _kHitTime = 0.05f; //50ms
    private const float _kAttackTime = 0.08f; //80ms
```

▼Judge.cs参照

```
Assembly-CSharp
Judge
195 // <summary>
196 // ノーツの判定処理
197 // </summary>
198 // <param name="timeLag">たたいた時間の誤差</param>
199 // <param name="noteIndex">対象ノーツのインデックス</param>
200 // 1 個の参照
201 void Judgement(float timeLag, int noteIndex)
202 {
203     //判定範囲外の場合は処理をしない
204     if (timeLag > _kAttackTime) return;
205
206     //インデックスの範囲チェック
207     if (!IsValidNoteIndex(noteIndex)) return;
208
209     //シングルノーツ以外は処理をしない
210     int noteType = _notesManager.noteType[noteIndex];
211     if (noteType != _kNoteTypeSingle) return;
212
213     JudgeType judgeType;
214
215     //判定の決定
216     if (timeLag <= _kCriticalTime + GManager.instance.timingOffset * 0.01f)
217     {
218         //Critical
219         judgeType = JudgeType.CRITICAL;
220     }
221     else if (timeLag <= _kHitTime + GManager.instance.timingOffset * 0.01f)
222     {
223         //Hit
224         judgeType = JudgeType.HIT;
225     }
226     else if (timeLag <= _kAttackTime + GManager.instance.timingOffset * 0.01f)
227     {
228         //Attack
229         judgeType = JudgeType.ATTACK;
230     }
231     else
232     {
233         return;
234     }
235     HandleHit(judgeType, noteIndex);
236 }
```

技術紹介⑤ : マネージャー(GManager)



ゲーム全体の進行やスコア、判定、設定などを一元管理するクラスを実装しました。

Singletonパターンを用いることで、各クラスから安全にアクセスできる構成にしています。
また、ゲーム状態(開始・ポーズ・終了・死亡)を明確に管理し、安定した進行制御を実現しました。

▼GManager.cs参照

```
4
5  /// <summary>
6  /// ゲーム全体の進行を管理するクラス
7  /// SingletonMonoBehaviour を継承
8  /// どのクラスにもアクセス可能にする
9  /// </summary>
10  /// Unity スクリプト (1 件のアセット参照) 199+ 個の参照
11  public class GManager : MonoBehaviour<GManager>
12  {
13      [Header("進行管理")]
14      public int songID; //曲のID
15      public float notesSpeed; //ノーツスピード
16      public float timingOffset; //タイミングオフセット
17      public bool isStart; //ゲームが開始したかどうかのフラグ
18      public float startTime; //ゲーム開始時間
19      public bool isDeath; //死亡したかどうかのフラグ
20      public bool isEnd; //ゲームが終了したかどうかのフラグ
21
22      [Header("スコア関連")]
23      public int score;
24      public float maxScore;
25      public float ratioScore; //スコアの比率
26
27      [Header("コンボ数関連")]
28      public int combo;
29      public int maxCombo;
30
31      public bool isComboBrokenOnce; //コンボが途切れたかどうかのフラグ
32
33      [Header("判定")]
34      public int critical;
35      public int hit;
36      public int attack;
37      public int miss;
38      public int coin;
39      public int maxCoin;
40
41      [Header("音量")]
42      public float masterVolume;
43      public float bgmVolume;
44      public float seVolume;
45      public float musicVolume;
```


技術紹介⑥ : データベース(1 / 2)



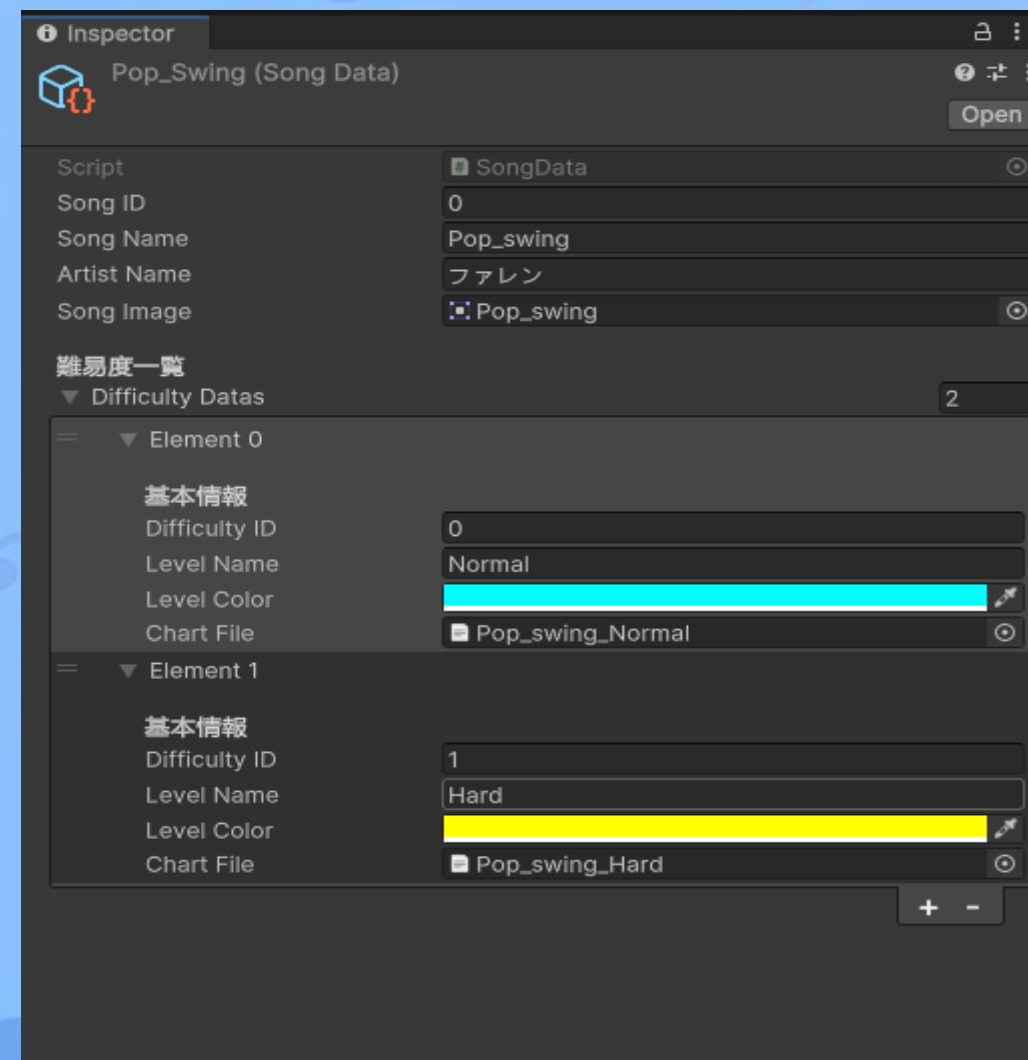
楽曲ごとの情報(譜面・画像、曲名など)を
ScriptableObjectとして管理し、
新規楽曲情報追加時にコード修正なしでデータ
追加のみで対応できる設計にしました。

これにより、制作効率と拡張性を大きく向上させました。

▼実際のゲーム画面



▼曲のデータを入れる項目



技術紹介⑥ : データベース(2 / 2)

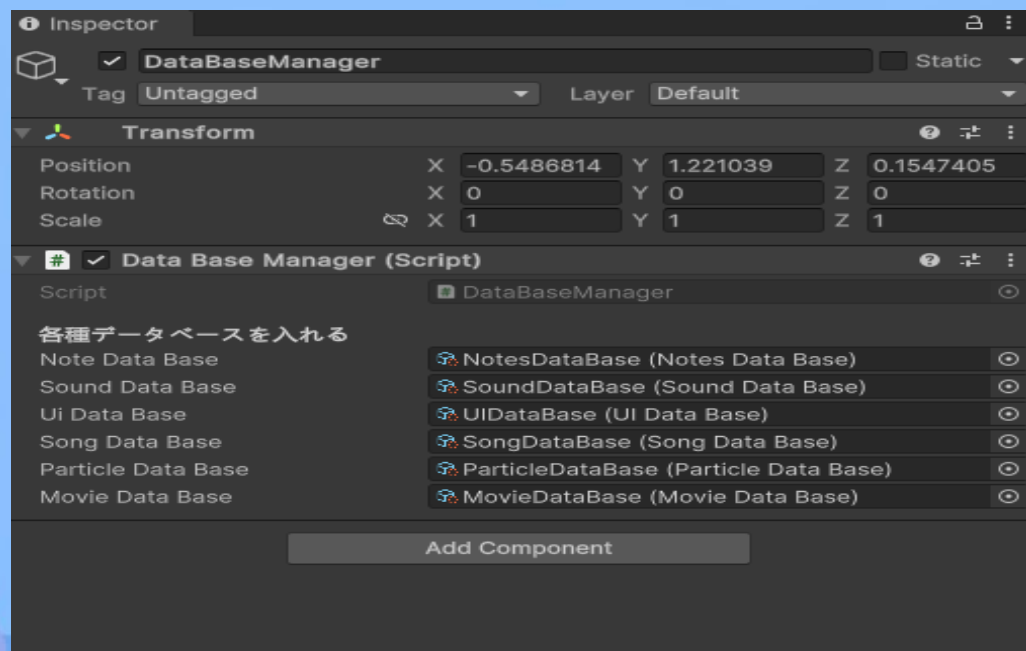


各データベースを一元管理するクラス

DataBaseManagerを実装しました。

Singletonパターンを用いることで、各クラスから個別にデータベースを参照する必要がなく、**インスペクター上でデータを設定する**だけで利用する設計にしています。

▼各種データベースを入れる項目



▼DataBaseManager.cs参照

```
using System;
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

/// <summary>
/// ゲーム全体で使うデータベースを一括管理するクラス
/// </summary>
/// Unity スクリプト (1 件のアセット参照) 116 個の参照
public class DataBaseManager : SingletonMonoBehaviour<DataBaseManager>
{
    [Header("各種データベースを入れる")]
    public NotesDataBase noteDataBase; // ノーツ関連のデータ
    public SoundDataBase soundDataBase; // サウンド関連のデータ
    public UIDataBase uiDataBase; // UI関連のデータ
    public SongDataBase songDataBase; // 曲関連のデータ
    public ParticleDataBase particleDataBase; // パーティクル関連のデータ
    public MovieDataBase movieDataBase; // Movie関連のデータ

    // 外部からのプロパティ読み込み可能
    0 個の参照
    public NotesDataBase noteDB => noteDataBase;
    2 個の参照
    public SoundDataBase soundDB => soundDataBase;
    5 個の参照
    public UIDataBase uiDB => uiDataBase;
    4 個の参照
    public SongDataBase songDB => songDataBase;
    1 個の参照
    public ParticleDataBase particleDB => particleDataBase;
    0 個の参照
    public MovieDataBase movieDB => movieDataBase;

    10 個の参照
    protected override void Awake()
    {
        base.Awake();
    }
}
```

技術紹介⑦ : シェーダー(1 / 2)



ロード画面の演出として、ストライプ模様が任意の角度で流れるアニメーションをシェーダーで再現しました。

滑らかな演出を再現し、プレイヤーの待機時間のストレス軽減を意識しています。

▼実際のロード画面

※ゲーム画面では流れてます!



技術紹介⑦ : シェーダー(2 / 2)



← ロード画面の説明の続き...

UV座標をベクトル方向に射影し、
時間によるスクロールを加えることで
軽度なアニメーションを実現しました。

▼BG_Load.shader参照

```
61         return o;
62     }
63
64     fixed4 frag (v2f i) : SV_Target
65     {
66         //角度 → ラジアン
67         float rad = radians(_Angle);
68
69         //方向ベクトル
70         float2 dir = float2(cos(rad),sin(rad));
71
72         //斜め方向の座標
73         float stripeCoord =
74             dot(i.uv,dir) * _Density
75             + _Time.y * _Speed;
76
77         //周期化
78         float stripe = frac(stripeCoord);
79
80         //アンチエイリアス
81         float aa = fwidth(stripe);
82
83         //ストライプマスク
84         float mask = smoothstep(
85             _Width - aa,
86             _Width + aa,
87             stripe
88         );
89
90         return lerp(_ColorA,_ColorB,mask);
91     }
92 }
93 ENDLOG
```

作品総括

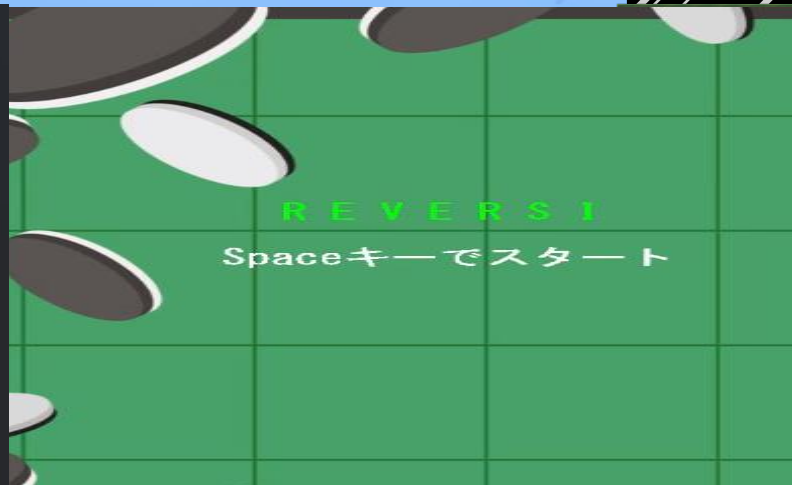
まだまだ **遊び要素が少ない!!**

上記の問題を解決するためには

- ・ 曲を増やす
- ・ 操作性の改善
- ・ ギミックに合わせた演出の強化

その他制作物

前半 チーム作品(1作品)
後半 個人作品(4作品)



チーム制作

2025年7月中旬～8月下旬

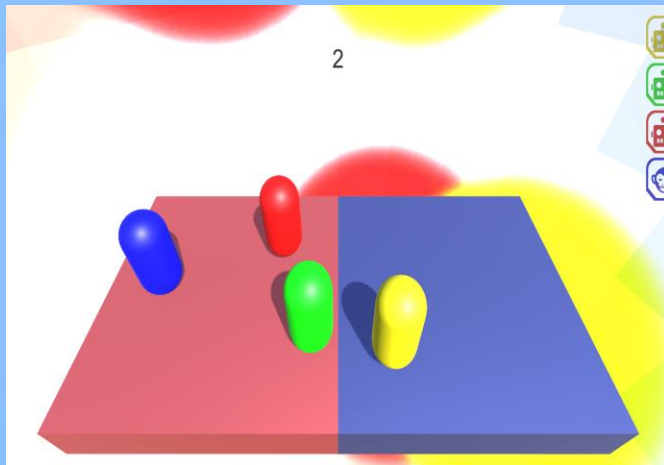
作品名 Island Party
ジャンル パーティゲーム
開発環境 C#、Unity
制作人数 4人
制作期間

2025年7月中旬～8月下旬

担当箇所 ミニゲーム部分のプログラム

頑張ったこと、学んだこと

- ・初めてのチーム制作
- ・チームでの連携
- ・コミュニケーションの大切さ



個人制作1

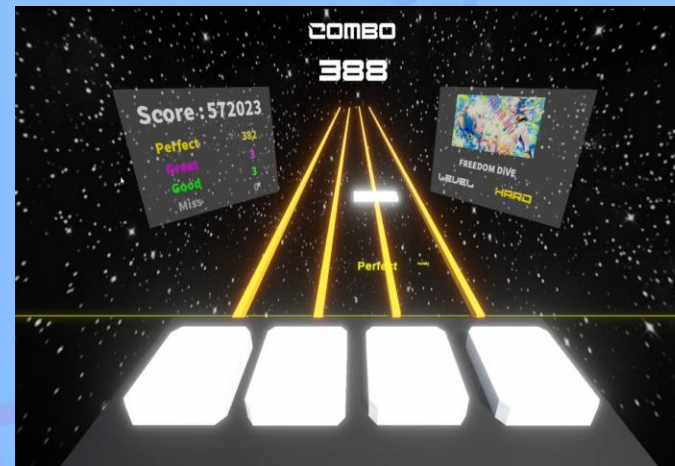
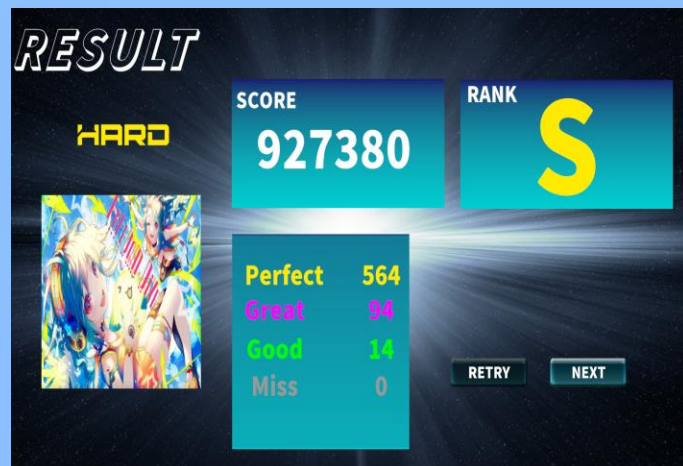
2025年8月～9月序旬

作品名 OTONITHM
ジャンル 3Dリズムゲーム
開発環境 C#、Unity
制作人数 1人
制作期間

2025年8月～9月序旬

頑張ったこと、学んだこと

- ・シェーダーの実装
- ・ポーズ画面の実装
- ・見た目のこだわり



個人制作2

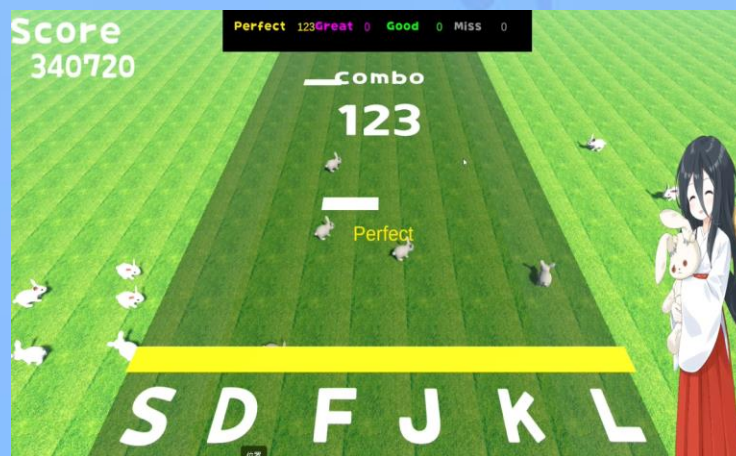
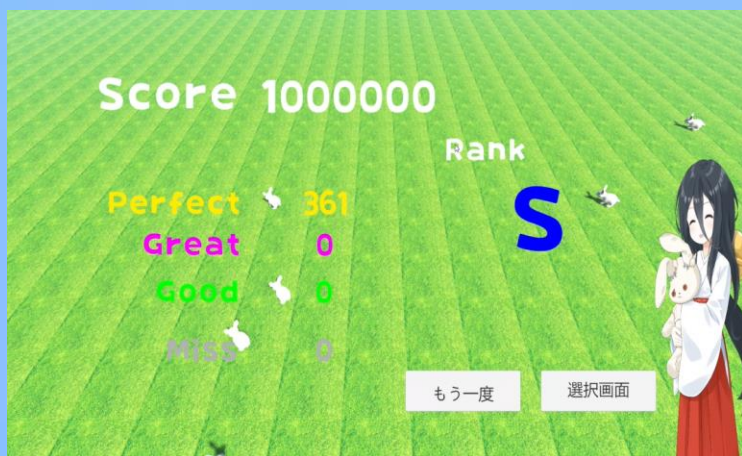
2025年6月～7月中旬

作品名 Rabbit × Music
ジャンル 3Dリズムゲーム
開発環境 C#、Unity
制作人数 1人
制作期間

2025年6月～7月中旬

頑張ったこと、学んだこと

- ・ データ管理
- ・ 外部からの譜面データの読み込み
- ・ ノーツの制御



個人制作3

2025年2月中旬～3月序旬

作品名 Jumping Action
ジャンル 2Dアクション
開発環境 DXライブラリ
制作人数 1人
制作期間

頑張ったこと、学んだこと

- ・ プレイヤーと敵の当たり判定
- ・ オブジェクト指向
- ・ ゲーム制作の大変さ

2025年2月中旬～3月序旬



個人制作4

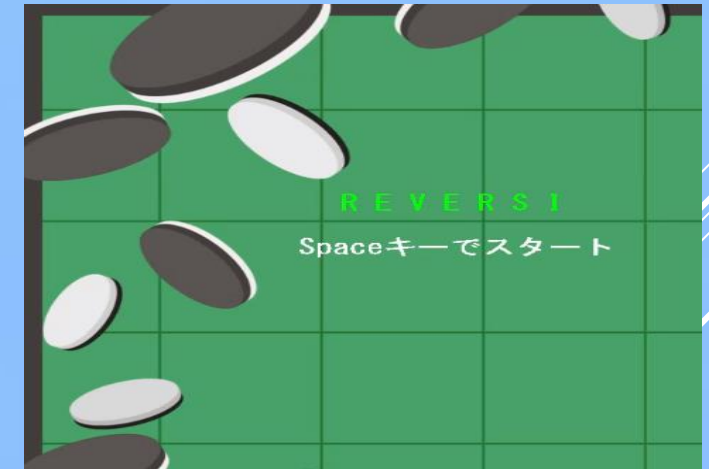
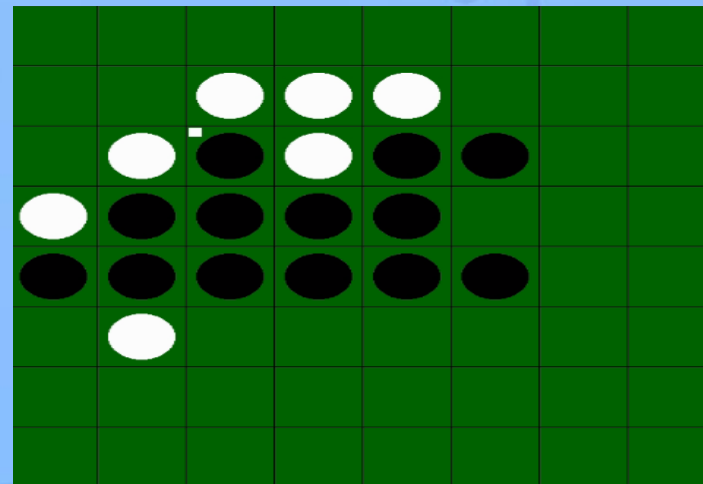
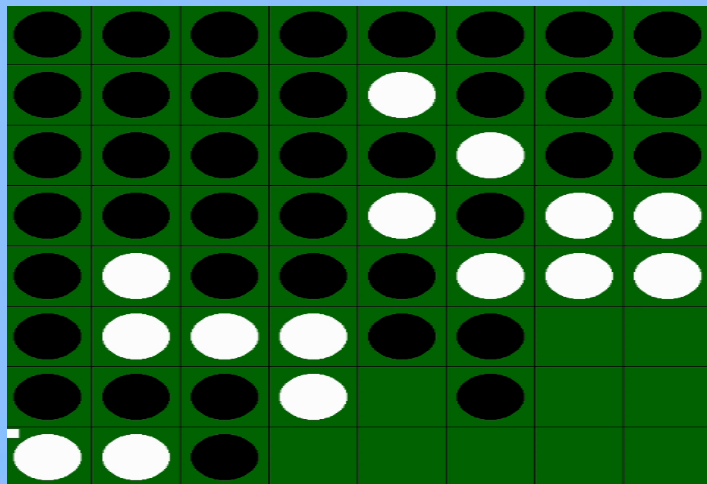
2025年1月序旬～下旬

作品名	REVERSI
ジャンル	2Dボードゲーム
開発環境	DXライブラリ
制作人数	1人
制作期間	

2025年1月序旬～下旬

頑張ったこと、学んだこと

- ・初めての2Dゲーム制作
- ・ゲーム制作の大変さ
- ・コード設計
- ・基本的なゲーム制作の流れ



今後の目標

私はプレイヤーの没入感を高められるゲームプログラマーになりたいと考えています。

そのため、処理の正確さやレスポンスの速さを追求し、快適なプレイ体験を提供する技術を磨いていきたいです。